



Tvorba internetových aplikácií

KI FMFI UK Bratislava

ferkom@dcs.fmph.uniba.sk

ostertag@dcs.fmph.uniba.sk

Jake Weary

- born
 - Jacob Weary
 - february 14, 1990
 - Trenton, New Jersey, U.S.
- occupation
 - actor
 - singer, musician
 - music producer, songwriter
- years active
 - 2004 – present
- known for
 - It Follows (2014)
 - Zombeavers (2014)
 - Altitude (2010)



Photo by Jennifer Pearl – © Jennifer Pearl

jQuery

- JavaScript library
 - fast & small
 - easy-to-use
 - feature-rich
 - cross browser compatible
- greatly simplifies basic JS tasks
 - HTML DOM traversal
 - HTML DOM manipulation
 - Events
 - Animations
 - AJAX

If I have seen further, it is by standing on the shoulders of giants.

Isaac Newton



Source: Wikipedia

Advantages

- Open-source, permissive MIT/GPL license
 - <https://tldrlegal.com/license/mit-license>
- Easily extensible, Powerful
- Small file size (< 99 kB)
- Browser support:
 - Chrome, Firefox, Safari, Opera, ...
 - jQuery 1.x (≈ 95 kB): IE ≥ 6 supported
 - jQuery 2.x (≈ 82 kB): IE < 9 not supported
 - saves 13% of size by dropping old browser support

For development

- Download: <http://jquery.com/download/>
- API docs: <http://api.jquery.com/>
- Plugins: <http://plugins.jquery.com/>
- jQueryUI: <http://jqueryui.com/>

Installation

- Simply link to a jQuery script in the HTML using a `<script>` tag:
 - `<script src="http://URL/jquery-1.11.0.min.js">
</script>`
- Either local file or on an external CDN server
 - <http://code.jquery.com/>
- Exposes jQuery to all other scripts

Usage

- Referring to the script gives access to the global jQuery (\$) function
- Everything with jQuery is done through this object
- \$(selector) chooses all HTML elements that are selected and returns a jQuery object
- \$.function executes a jQuery function without HTML elements passed

Selectors 1

- CSS like selectors to filter HTML nodes by:
 - Element (img, table, header, ...)
 - ID (#navigation, #footer, ...)
 - Class (.currentItem, .selected, ...)
 - Child selectors (:first-child, :nth-child, ...)
 - Child selector (parent > child)
 - Descendant selector (ancestor descendant)
 - Attribute ([href], [value="password"], ...)
 - ...

Selectors 2 – example

`$("body nav#main a.current:nth-child(2)")`

```
<body>
  <nav id="main">
    <a href="" class="current">
      Home
    </a>
    <a href="" class="current">
      Contact
    </a>
  </nav>
</body>
```

Selectors 3

- Very powerful traversal through the HTML DOM
- Do not depend on the browser support of CSS selectors
- Much easier than `document.getElementById...`

Modifying DOM 1/3

- We want to be able to
 - Add/remove/get HTML nodes
 - Add/remove/get node content
 - Add/remove/get node attributes
 - Add/remove/get node CSS settings

Modifying DOM 2/3

```
var cssWidth = $("body").css('width');

$("body").html('<h1>Hello world!</h1>');
$("body").css('background-color', 'yellow');
$("body").css({
    'color':      'red',
    'font-size':  '25px',
    'padding':    '0.5em'
});
```

Modifying DOM 3/3

```
$("#div").addClass("MyClass");  
$("#div").removeClass("current");  
$("#h1").toggleClass("selected");
```

```
var myHref = $("#a.current").attr("href");  
$("#img").attr("src", myHref);
```

```
var nodeAsText = $("#p").text();  
$("#p").text("<b>Some</b> new text.");
```

Events

<u>Mouse</u>	<u>Keyboard</u>	<u>Form</u>	Document or Window
click	keydown	submit	load
dblclick	keyup	change	resize
mouseenter	keypress	focus	scroll
mouseleave		blur	unload

jQuery event listeners 1/3

HTML:

```
<table>
  <tr><td>1</td><td>6</td><td>2</td><td>7</td></tr>
  <tr><td>4</td><td>9</td><td>5</td><td>0</td></tr>
</table>
<hr/>
<div id="list"></div>
```

CSS:

```
td {
  background-color: yellow;
  padding: 0.31em;
  font-size: 2em;
}
#list span {
  background-color: lightblue;
  margin: 0.1em;
  padding: 0.3em;
}
```

JavaScript:

```
$("#table").on("click", "tr", function() {
  var text = $(this).text();
  $("#list").append(
    "<span>" + text + "</span>"
  );
});
```

Result:

1	6	2	7	
4	9	5	0	
4950	1627	4950	1627	1627

jQuery event listeners 2/3

```
$("#img").click(function() {  
    alert("clicked on an image");  
});
```

```
$("#a").click(false);
```

```
$("#a").click(function() {  
    return false;  
});
```

```
$("#a").click(function(event) {  
    event.preventDefault();  
})
```

```
$(document).ready(function() {  
    alert("document ready");  
});
```

```
$(function() {  
    alert("document ready");  
});
```


jQuery event listeners 3/3

```
$("#nav a").hover(  
    function () { $(this).addClass("highlighted"); },  
    function () { $(this).removeClass("highlighted"); }  
);
```

```
$("#nav a").hover(  
    function () { $(this).toggleClass("highlighted"); }  
);
```

Firing events

- `$("a").click();`
- `$("header").dblclick();`
- `$("img").mouseenter();`
- `$("p").mouseleave();`
- `$("textarea").keydown();`
- `$("textarea").keyup();`

Animations

- Easily animate almost all CSS properties
- Nice and smooth animations
- Choice of duration and easing
- Few exceptions:
 - Colors
 - but available in jQueryUI
 - Non-numerical attributes are hard to animate
- Display property, background-image, ...

Animating content

```
$("#img").animate({"opacity" : "0.5"}, 1000);
```

```
$(function () {  
    $("#img").animate({  
        "margin-top": "200px",  
    }, {  
        duration: 500,  
        easing: "linear",  
        complete: function () { alert("Done."); },  
        step: function (now, fx) {  
            var data = fx.elem.id + " " + fx.prop + ": " + now;  
            $("#body").append("<div>" + data + "</div>");  
        }  
    });  
});
```

Animation handling

```
$("#img").animate({"opacity" : "0.0"}, 1000)  
    .delay(100)  
    .animate({"opacity" : "1.0"}, 1000);  
  
$("#img").stop();
```

AJAX

- Asynchronous JavaScript and XML
- Send server requests without refreshing the page
- Client JavaScript receive server response in:
 - Plain text
 - XML
 - JSON

AJAX usage

- Interactive form validation
- More responsive pages
- Easier and faster administration of pages
- Facebook and Google use AJAX **a lot**
- HTML5 games
- Chat applications
- ...

AJAX without jQuery

```
var xhr;
if (window.XMLHttpRequest) {
    xhr = new XMLHttpRequest();
} else { // code for IE6, IE5
    xhr = new ActiveXObject("Microsoft.XMLHTTP");
}
xhr.onreadystatechange = function() {
    if (xhr.readyState==4 && xhr.status==200)
        document.getElementById("myDiv").innerHTML =
            xhr.responseText;
}
xhr.open("GET","ajax_info.txt", true);
xhr.send();
```


AJAX with jQuery – sending data

```
$.ajax({  
    type:    "POST",  
    url:     "save.php",  
    data:    { name: "John", location: "Boston" }  
}).done(function( msg ) {  
    $("div.info").text(msg);  
});
```

// server side – file save.php

```
<?php
```

```
$name = $_POST["name"]; $location = $_POST["location"];
```

```
date_default_timezone_set('Europe/Bratislava');
```

```
$date = date('g:i:s', time());
```

```
echo "Time: $date; Name: $name; Location: $location";
```

```
?>
```

AJAX with jQuery – receiving data

- retrieve the **latest** version of an HTML page:

```
$.ajax({  
    url:      "test.html",  
    cache:   false  
}).done(function( html ) {  
    $("#results").append(html);  
});
```

- Load and **execute** a JavaScript file:

```
$.ajax({  
    type:      "GET",  
    url:      "test.js",  
    dataType:  "script"  
});
```

AJAX with jQuery – communicate

```
var menuId = $("ul.nav").first().attr("id");
var request = $.ajax({
    url:      "script.php",
    type:     "POST",
    data:     {id : menuId},
    dataType: "html"
});
request.done(function(msg) {
    $("#log").html(msg);
});
request.fail(function(jqXHR, textStatus) {
    alert("Request failed: " + textStatus);
});
```

The X in AJAX?

- Data sent to server can be encoded as XML
 - The server parses the XML
 - Based on received data creates an XML response
 - Sends to client
 - Accessible through jQuery done function which can get a XHR object

XML response

```
$.ajax({  
    url: "process.php",  
    type: "POST",  
    async: false,  
    data: xmlDomObject,  
    processData: false,  
    dataType: "xml"  
}).done(function(data, textStatus, xhr) {  
    // data contains a complete XML document  
});
```

JSON

- JavaScript Object Notation
- Newer alternative for XML
 - Not only concerning AJAX
- Direct support in jQuery AJAX calls
 - `dataType : "json"`
 - Automatically converts received/sent data to JSON

JSON Example

```
{  
  "firstName": "John",  
  "lastName": "Smith",  
  "age": 25,  
  "address": {  
    "streetAddress": "21 2nd Street",  
    "city": "New York",  
    "state": "NY",  
    "postalCode": 10021  
  },  
  "phoneNumber": [  
    {  
      "type": "home",  
      "number": "212 555-1234"  
    },  
    {  
      "type": "fax",  
      "number": "646 555-4567"  
    }  
  ]  
}
```

JSON with AJAX

- Client side
 - `JSON.stringify()` to convert a JavaScript object to JSON
 - `JSON.parse()` to convert a JSON string to a JavaScript object
 - Automatic with jQuery
- Server side (PHP)
 - `json_encode()`
 - `json_decode()`

Watch out

- Accessing *this* in anonymous functions

```
function MyClass() {  
    var self = this;  
    $("p").each(function() {  
        // this refers to the paragraph  
        // self refers to the class instance  
    })  
}
```

- Spaces in selectors

```
$("#p .red")  
$("#p.red")
```

jQuery.extend()

- **provides substitution for mixins or multiple inheritance in JavaScript**
- ```
var object1 = { apple: 0, cherry: 97,
 banana: { weight: 52, price: 100 }
};
var object2 = { banana: { price: 200 },
 durian: 100
};
var result = {};
```
- shallow merge of object1 and object2 into result
  - `$.extend(result, object1, object2);`  
`$("#log1").append(JSON.stringify(result));`
- deep merge of object1 and object2 into empty object (result is also return value)
  - `result = $.extend(true, {}, object1, object2);`  
`$("#log2").append(JSON.stringify(result));`
- HTML

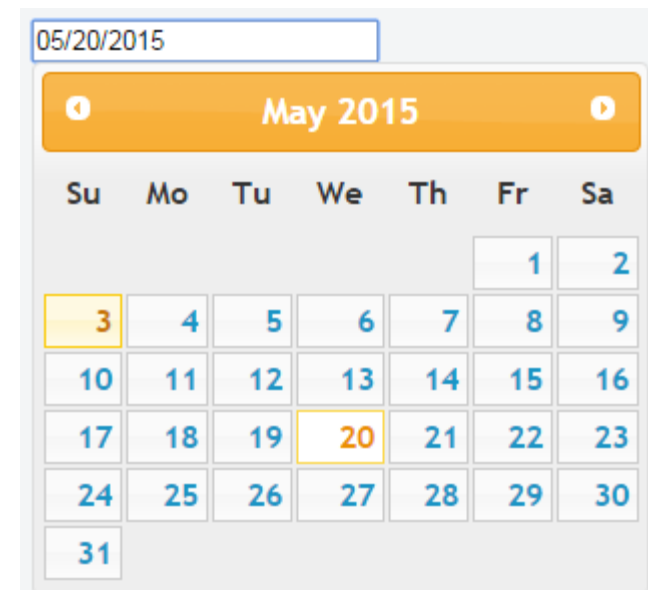
```
<div id="log1"></div>
<div id="log2"></div>
```
- result:
  - `{"apple":0,  
 "banana":{"price":200},  
 "cherry":97,  
 "durian":100}`
  - `{"apple":0,  
 "banana":{"weight":52,"price":200},  
 "cherry":97,  
 "durian":100}`
- <https://jsfiddle.net/xd3v1yag/>

# jQuery UI

- Great plugin for jQuery
- Various themable production-ready UI widgets
- Extensions to jQuery functionality
  - Animation of color and style classes
  - Hide and show effects
  - 30 additional easings
    - jQuery has only linear and swing
  - Add interactions (like draggable, resizable, ...)
  - Additional selectors
    - :data() – elements with data stored under the specified key
    - :focusable – elements which can be focused
    - :tabbable – elements which the user can focus via tabbing

# Using jQuery UI on a Web Page

- In head reference theme, jQuery and jQuery UI:
  - `<link rel="stylesheet" href="jquery-ui.min.css">`
  - `<script src="jquery.js"></script>`
  - `<script src="jquery-ui.min.js"></script>`
- HTML:  
Date:  
`<input type="text" id="date">`
- JavaScript:  
`$("#date").datepicker();`



# How jQuery UI Works

- Widgets are built on top of the Widget Factory
  - Provides the same general API to all of them
- Initialization
  - `$("#elem").progressbar();`
    - All options are set to default values
  - `$("#elem").progressbar({ value: 20 });`
    - Pass as many or as few options as needed
    - Remaining options will just use their default values
  - Options can set after initialization as well
    - Use the option method

# jQuery UI – methods

- After initialization, we can
  - query widget state
  - perform actions
- Everything takes the form of a method call, e.g.
  - Plugin invocation
    - call the value method on progressbar
      - `$("#elem").progressbar("value");` `// plugin invocation`
    - if the method accepts parameters, pass them after the method name:
      - `$("#elem").progressbar("value", 40);`
    - method chaining is available
  - Instance invocation:
    - `var elem = $("#elem").progressbar();`
    - `pb = elem.progressbar("instance");`
    - `pb.value(80);` `// instance invocation`

# jQuery UI – common methods

- option (change options after initialization )
  - `$("#elem").progressbar("option", "value", 30);`
- disable, enable
  - `$("#elem" ).progressbar("disable" or "enable");`
- destroy (destroy widget and return back to the original markup)
  - `$("#elem").progressbar("destroy");`
  - the widget will automatically destroy itself if you're removing the element
    - directly via `.remove()`
    - by modifying an ancestor with `.html()` or `.empty()`
- widget (returns wrapper element)
  - in case of progressbar original element is returned
    - not all widgets use wrapper elements

# jQuery UI – events

- Widgets have events associated with their behaviors
  - The events names are prefixed with the widget name
    - `$("#elem").bind("progressbarchange", function() {  
    alert( "The value has changed!" );  
});`
  - Each event has a corresponding callback exposed as an option
  - bind to the progressbarchange event or hook into progressbar's change callback (it's the same)
    - `$( "#elem" ).progressbar({  
    change: function() {alert( "The value has changed!" );}  
});`
- Common event
  - create
    - Triggered immediately after the widget is created



# jQuery UI – interactions

- jQuery UI provides set of mouse-based interactions
  - building blocks for rich interfaces
- Draggable
  - Allows elements to be moved using the mouse.
- Droppable
  - Creates targets for draggable elements.
- Resizable
  - Allows changing the size of an element using the mouse.
- Selectable
  - Allows selecting elements individually or in a group.
- Sortable
  - Allows reordering of elements in a list or grid.

# Sortable

- HTML:

```
<ul id="sortable">
 <li class="ui-state-default">Item 1
 <li class="ui-state-default">Item 2
 <li class="ui-state-default">Item 3
 <li class="ui-state-default">Item 4

```

- JavaScript: `$("#sortable").sortable();`

- <https://jsfiddle.net/RiOS/jfgz1vw2/>
- <http://jqueryui.com/sortable/>

# Draggable

- HTML:  

```
<div id="draggable" class="ui-widget-content">
 Drag me</div>
<div id="droppable" class="ui-widget-header">
 Drop here</div>
```
- JavaScript:  

```
$("#draggable").draggable();
$("#droppable").droppable({
 drop: function(event, ui) {
 $(this).addClass("ui-state-highlight").html("Done!");
 }
});
```
- <https://jsfiddle.net/RiOS/xtcugtef/>
- <http://jqueryui.com/droppable/>

# Dialog Widget

- HTML:  
`<div id="delete" title="Empty the recycle bin?">  
 Permanently delete all items?  
</div>  
<div id="main">Main page.</div>`
- JavaScript:  
`$("#delete").dialog({ modal: true,  
 buttons: {  
 "Delete all": function() { $("#main").html("-- deleted --");  
 $(this).dialog("close"); },  
 Cancel: function() { $(this).dialog("close"); }  
 }  
});`
- <https://jsfiddle.net/RiOS/gzzciv43/>
- <http://jqueryui.com/dialog/>

# Autocomplete Widget

- HTML:  

```
<div class="ui-widget">
 <label for="tags">Tags: </label><input id="tags">
</div>
```
- JavaScript:  

```
var langs = ["ActionScript", "AppleScript", "C",
"C++", "Clojure", ..., "Scala", "Scheme"];
$("#tags").autocomplete({ source: langs });
```
- <https://jsfiddle.net/RiOS/58oxwtcg/>
- <http://jqueryui.com/autocomplete/>

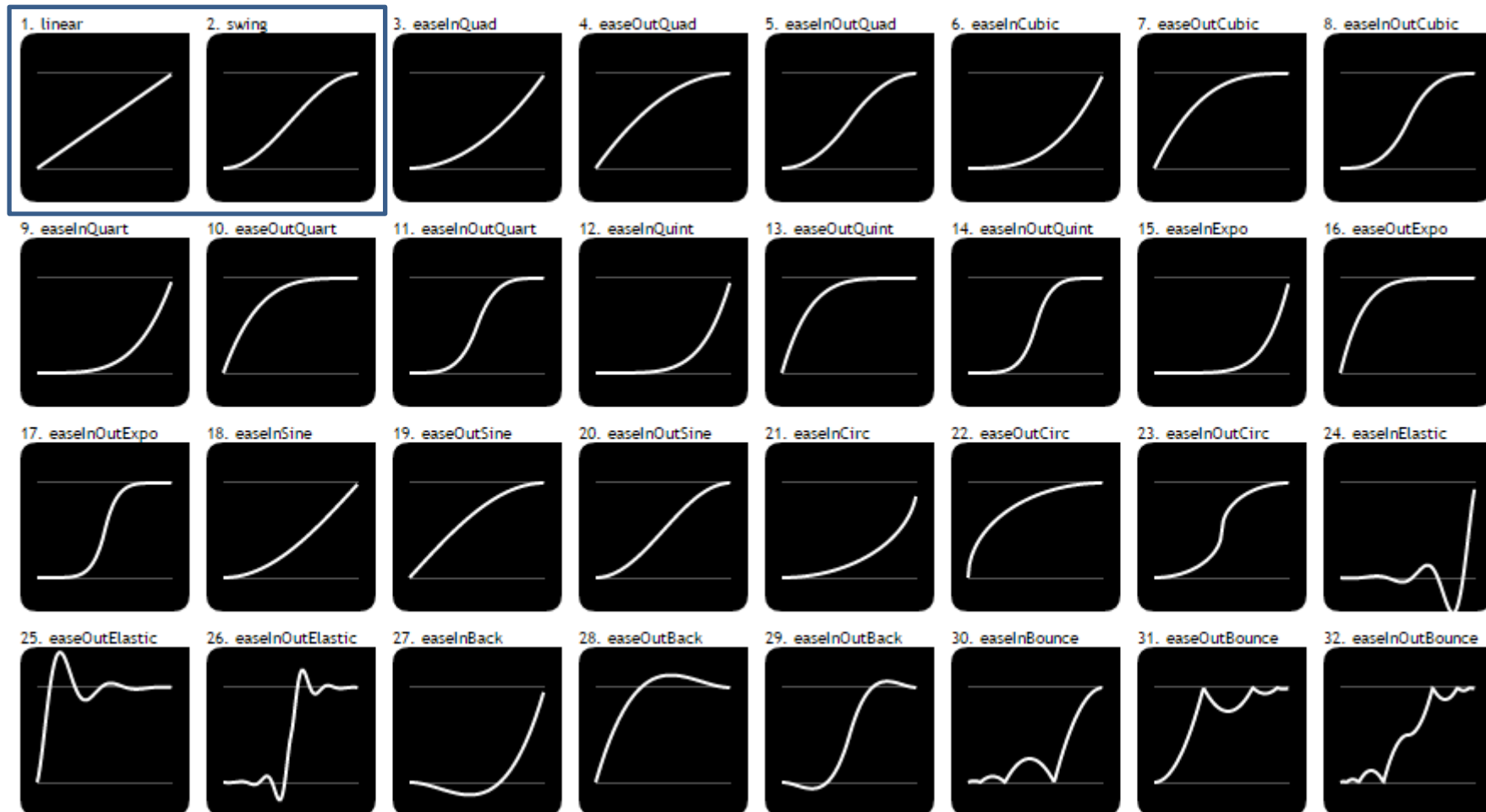
# Slider Widget

- HTML:  

```
<p> <label for="amount">Price range:</label>
 <input type="text" id="amount" readonly> </p>
<div id="slider-range"></div>
```
- JavaScript:
- ```
function setAmount(min, max)  
{ $("#amount").val("$" + min + " to $" + max); }
```
- ```
$("#slider-range").slider({
 range: true, values: [75, 300], min: 0, max: 500,
 slide: function(e, ui) {setAmount(ui.values[0], ui.values[1]); }
});
var slider = $("#slider-range").slider("instance");
setAmount(slider.values(0), slider.values(1));
```
- <https://jsfiddle.net/RiOS/Lkqvrnzd/>
- <http://jqueryui.com/slider/>

# jQuert UI – additional easings

## Default easings



# Summary

- Great JavaScript library
- Simplified otherwise painful tasks
- Wide variety of plugins
- Hundreds of galleries/sliders/widgets based on jQuery
- Once jQuery, always jQuery :-)
  - but there are alternatives like <http://zeptojs.com/>



# You might not need jQuery :-)

- <http://youmightnotneedjquery.com/>
- Native selectors are match faster (2 to 4 times)
- jQuery is 🐌

